# Big data processing and analytics

Antonio Villella

Anno accademico 2026–2027

# Indice


**1 Introduzione ai Big Data** **6**
- 1.1 Che cosa sono i Big Data? . . . 6
- 1.2 Fonti e Generatori di Big Data . . . 6
- 1.3 Le "V" dei Big Data . . . 7
- 1.4 Framework Big Data: Motivazioni e Sfide . . . 7
  - 1.4.1 Perché abbiamo bisogno di nuovi framework? . . . 7
  - 1.4.2 Il principio della Data Locality . . . 7
  - 1.4.3 Architettura dei Data Center e Scalabilità . . . 8
  - 1.4.4 Le sfide tecniche . . . 8
  - 1.4.5 Il problema tipico nei Big Data . . . 8

**2 Big Data Architectures** **9**
- 2.1 Introduzione alle Architetture Big Data . . . 9
  - 2.1.1 Quando utilizzare un'architettura Big Data . . . 9
- 2.2 La Lambda Architecture . . . 9
  - 2.2.1 Definizioni a confronto . . . 10
  - 2.2.2 Il Modello dei Dati nella Lambda Architecture . . . 10
  - 2.2.3 Requisiti dell'Architettura . . . 11

**3 Hadoop e MapReduce** **12**
- 3.1 Introduzione al Framework Hadoop . . . 12
- 3.2 Il Principio della Data Locality . . . 13
- 3.3 Campi di Applicazione di MapReduce . . . 13
- 3.4 Il Paradigma di Programmazione MapReduce . . . 14
  - 3.4.1 Struttura Dati: Coppie Chiave-Valore . . . 14
  - 3.4.2 Le Fasi di Elaborazione . . . 14
- 3.5 Esempio Pratico: Il Problema del Word Count . . . 14

**4 Sviluppo di applicazioni MapReduce in Hadoop** **16**
- 4.1 Architettura di un'applicazione MapReduce e Terminologia . . . 16
  - 4.1.1 Il livello di parallelizzazione . . . 17
- 4.2 Componenti e Flusso dei Dati . . . 17
  - 4.2.1 Il Driver . . . 17
  - 4.2.2 Il Mapper . . . 17
  - 4.2.3 Il Reducer e il partizionamento dei dati (Shuffle and Sort) . . . 17
- 4.3 Tipi di Dato (Data Types) in Hadoop . . . 18
- 4.4 Formati di Input e Output . . . 18
- 4.5 Struttura base del codice Hadoop: Word Count Example . . . 19
  - 4.5.1 Implementazione del Driver . . . 19
  - 4.5.2 Implementazione del Mapper . . . 20

4.5.3 Implementazione del Reducer . . . . . 21
4.6 Ottimizzazione delle Prestazioni: Il Combiner . . . . . 21
4.6.1 Perché nel Word Count si sommano i valori anziché contarli? . . . . . 22
4.6.2 Implementazione e Dichiarazione . . . . . 22
4.7 Creazione di Tipi di Dato Personalizzati . . . . . 22
4.8 Condivisione di Parametri e Contatori . . . . . 23
4.8.1 Passare parametri al Mapper e al Reducer . . . . . 23
4.8.2 I Contatori (Counters) . . . . . 24
4.9 Job "Map-Only" e ottimizzazioni avanzate . . . . . 24
4.9.1 Map-Only Job . . . . . 24
4.9.2 In-Mapper Combiner: un passo oltre il Combiner . . . . . 24
4.10 Pattern di Ottimizzazione e Filtraggio in MapReduce . . . . . 25
4.11 Ottimizzazione tramite In-Mapper Combiner (Esercizio 9) . . . . . 25
4.11.1 Funzionamento dell'In-Mapper Combiner . . . . . 25
4.11.2 Limiti e Gestione della Memoria Principale . . . . . 26
4.11.3 Esempio di Dati (Toy Example) . . . . . 26
4.11.4 Analisi dell'Implementazione Java . . . . . 27
4.12 Filtraggio dei Dati e Passaggio Dinamico di Parametri (Esercizio 12) . . . . . 28
4.12.1 Il Pattern di Filtraggio (Map-Only Job) . . . . . 29
4.12.2 Passaggio di Parametri tramite l'Oggetto Configuration . . . . . 29
4.12.3 Esempio di Dati . . . . . 30
4.12.4 Analisi dell'Implementazione Java . . . . . 30

# Introduzione al Corso e Modalità d'Esame

## Informazioni Generali sul Corso

Il corso affronta le sfide e le opportunità legate all'analisi e alla gestione di enormi quantità di dati (Big Data). Vengono introdotti non solo modelli di dati per l'archiviazione (database relazionali e non relazionali scalabili), ma soprattutto i nuovi paradigmi di programmazione e i framework distribuiti necessari per l'elaborazione dei dati, come Hadoop e Spark.

### Docenti e Materiale Didattico

I docenti del corso sono Paolo Garza, Hadi Ibrahim e Lorenzo Innocenti. Per eventuali dubbi, i docenti sono a disposizione durante le pause o al termine della lezione, oppure tramite appuntamento via email.

Tutto il materiale di riferimento (slide, esercizi, attività di laboratorio ed esami passati) è disponibile sulla pagina web dedicata del corso su `minds.polito.it`. Le videolezioni sono rese disponibili sul Portale della Didattica. Sebbene vi siano dei libri di testo suggeriti (come *Spark: The Definitive Guide* o *Hadoop, The Definitive Guide*), le slide e gli esercizi forniti sono considerati sufficienti per l'apprendimento e il superamento dell'esame.

*Osservazione* 0.1 (Metodo di studio). L'impegno stimato per il corso prevede che per ogni credito formativo (CFU) si dedichino circa 10 ore in aula/laboratorio e 15 ore di studio personale (in totale 25 ore per CFU). Il professore consiglia vivamente di provare a risolvere gli esercizi e le attività di laboratorio in autonomia *prima* di consultare le soluzioni, al fine di assimilare i "pattern" fondamentali di risoluzione, necessari per costruire applicazioni complesse.

## Prerequisiti e Competenze Pregresse

Per affrontare il corso con successo sono richieste alcune competenze di base in ambito di programmazione e basi di dati:

- **Java:** Necessario per scrivere applicazioni MapReduce su Hadoop. È richiesta una conoscenza di base (es. cicli e operazioni elementari sulle stringhe).
- **Python:** Utilizzato per scrivere applicazioni su Spark. Anche in questo caso sono richieste operazioni di base (filtraggio, trasformazione dei dati e calcolo di statistiche).
- **Database e SQL:** Utile per comprendere il modello relazionale e, in particolar modo, per interfacciarsi con le API di Spark SQL.

## Programma e Argomenti del Corso

Il corso è strutturato per insegnare a scrivere applicazioni software (non solo l'architettura dei sistemi) ed è diviso in diversi macro-argomenti:

### Hadoop e MapReduce

Verrà studiata l'architettura base di Hadoop e il paradigma di programmazione MapReduce (in Java).

*Osservazione* 0.2 (Perché studiare ancora MapReduce?). Sebbene oggi non sia il framework più performante, il professore sottolinea che esistono ancora moltissimi sistemi legacy basati su questo paradigma (in alcuni casi con 50 anni di dati storici archiviati, la cui migrazione richiederebbe settimane o mesi, un po' come accade per i vecchi software in COBOL). È quindi un'abilità fondamentale per un ingegnere dei dati saper mettere mano, leggere e aggiornare questo tipo di codice.

### Apache Spark

Spark è attualmente uno dei framework più utilizzati per l'analisi di enormi quantità di dati. Il corso esplorerà la sua architettura e insegnerà a scrivere applicazioni utilizzando due diversi approcci:

- **RDD (Resilient Distributed Datasets):** La versione precedente delle API di Spark (scritta in Python/Java). Secondo il docente, gli RDD sono fondamentali da studiare non solo per via del codice legacy, ma anche perché permettono di comprendere esattamente quanti e quali dati viaggiano sulla rete, fattore chiave per valutare l'efficienza di un'applicazione Big Data.
- **Spark SQL e DataFrames:** Le API più recenti. Permettono di utilizzare direttamente il linguaggio SQL per processare i dati. Spesso, conoscendo già l'SQL, questo approccio risulta molto più semplice ed è il più utilizzato oggi in produzione.

### Librerie Avanzate: MLlib e Spark Streaming

- **MLlib (Machine Learning):** La libreria di machine learning scalabile di Apache Spark. Il professore precisa che l'obiettivo non è imparare a scrivere nuovi algoritmi di machine learning distribuiti da zero, ma imparare a usare quelli già presenti e parallelizzabili per analizzare grandi quantità di dati.
- **Spark Streaming:** Strumenti per l'analisi di flussi continui di dati. La transizione concettuale è semplice: una volta compreso come analizzare i dati "in batch" con Spark, l'approccio allo streaming risulta identico; cambia unicamente la sorgente di input (che passa dall'essere un grande file statico a un flusso continuo).

## Regole e Modalità d'Esame

L'esame consiste unicamente in una prova scritta erogata tramite piattaforma d'ateneo (Exam Platform con browser protetto *Lockdown*), della durata di **1,5 ore (90 minuti)**. Per sostenere l'esame è necessario portare il proprio PC.

Il punteggio massimo è di **31 punti**, così strutturato:

- **2 Esercizi di Programmazione (max 27 punti):** Basati sulla progettazione di programmi Hadoop/MapReduce e/o API Spark. Spesso un singolo programma richiederà di risolvere due sotto-problemi, ed è valutata l'abilità dello studente nel riutilizzare il codice in modo efficiente per entrambi.
- **2 Domande Teoriche a risposta multipla (max 4 punti):** Vertono su tutti gli argomenti (architetture, RDD, HDFS, ecc.). Spesso queste domande presentano porzioni di codice di cui viene richiesto di prevedere l'output.

*Osservazione* 0.3 (Esame a Libro Aperto). L'esame è "open book", ovvero è consentito consultare libri, appunti e materiale cartaceo stampato. **Tuttavia, non è permesso alcun dispositivo elettronico** (cellulari, tablet o altri PC) al di fuori di quello utilizzato per l'esame. Il docente avvisa severamente che qualsiasi tentativo di copiatura ("cheating") sarà sanzionato con l'annullamento dell'esame e la potenziale sospensione di un anno dagli appelli (con conseguente perdita di eventuali borse di studio).

# Capitolo 1

# Introduzione ai Big Data

## 1.1 Che cosa sono i Big Data?

Il termine **Big Data** è generalmente associato a enormi quantità di dati ("petabytes") raccolte in grandi collezioni. Tuttavia, il concetto non si limita alla sola quantità, ma coinvolge diverse dimensioni fondamentali:

- **Storage**: i dati devono essere memorizzati utilizzando approcci efficienti ed efficaci. Spesso non si utilizzano i tradizionali database relazionali, ma ci si affida a *distributed file system* o database NoSQL.
- **Analytics**: l'obiettivo finale è estrarre conoscenza e valore dai dati attraverso metodologie di analisi.
- **Technologies**: sono necessarie nuove tecnologie e framework per elaborare e analizzare questi dati in modo scalabile.

Inizialmente, i framework per i Big Data sono stati sviluppati dalle grandi aziende di internet (le "Big Tech" come Google e Meta). Oggi, tuttavia, la necessità di analizzare enormi moli di dati riguarda qualsiasi settore, dalle banche agli impianti industriali.

*Osservazione* 1.1. **Sfruttare i dati esistenti per nuovi task**
Il professore sottolinea che un buon *data scientist* o *data engineer* deve saper utilizzare i dati già a disposizione per risolvere nuovi problemi. Ad esempio, Google ha iniziato a raccogliere le query degli utenti per fornire risposte migliori, ma ha poi utilizzato gli stessi dati per fare *nowcasting* (previsione a brevissimo termine), stimando in tempo reale quante persone fossero affette da influenza in una determinata area.

## 1.2 Fonti e Generatori di Big Data

Esistono diverse fonti in grado di generare enormi quantità di dati, che tipicamente vengono poi convogliate e memorizzate in un singolo Data Center. Le principali categorie includono:

1. **User Generated Content (Web & Mobile)**: contenuti generati dagli utenti su piattaforme come Facebook, Instagram, Yelp, TripAdvisor, Twitter e YouTube.
2. **Health and scientific computing**: dati ad alta risoluzione provenienti dal settore medico, immagini satellitari (utili ad esempio per identificare aree allagate o bruciate), microscopi e telescopi.

3. **Log files**: file di testo generati da web server o sistemi automatici che tracciano eventi e accessi.
4. **Internet Of Things (IoT)**: reti di sensori (*Sensor networks*), RFID e *smart meters*. Anche se ogni singolo sensore invia una quantità minuscola di dati, l'aggregazione di miliardi di sensori genera moli di dati enormi.

## 1.3 Le "V" dei Big Data

Il contesto dei Big Data è tradizionalmente descritto attraverso le **3V**:

- **Volume (Scale of data)**: la scala dimensionale dei dati. L'unità di misura tipica entra nell'ordine dei Petabyte (PB). Intorno al 2012, Google processava oltre 100 PB di dati al giorno, mentre Facebook oltre 10 PB.
- **Variety (Different forms of data)**: l'eterogeneità dei dati in ingresso. Un sistema Big Data dovrebbe poter gestire testo, immagini, grafi e log.
- **Velocity (Analysis of streaming data)**: la velocità con cui i dati vengono generati e la necessità di analizzare flussi di dati continui (stream) elaborando milioni di eventi in pochi millisecondi.

A queste, nel tempo, sono state aggiunte altre due dimensioni:

- **Veracity (Uncertainty of data)**: la qualità dei dati.
- **Value**: la capacità di estrarre informazioni utili dal dato.

*Osservazione* 1.2. **Veracity e qualità del dato**
Il professore ricorda il principio *"Garbage in, garbage out"*: se l'input è rumoroso o di scarsa qualità, anche l'output degli algoritmi di Machine Learning sarà pessimo. Inoltre, i dati grezzi raramente sono pronti per l'analisi: in una tipica pipeline di Data Science, dal 60% al 70% del tempo è dedicato alla pulizia del dato (*data cleaning*) e alla sua trasformazione in vettori o matrici.

## 1.4 Framework Big Data: Motivazioni e Sfide

### 1.4.1 Perché abbiamo bisogno di nuovi framework?

Nel passato avevamo già a disposizione database relazionali efficienti, sistemi NoSQL e l'High-Performance Computing (HPC). Tuttavia, questi sistemi mostrano dei limiti con moli di dati eccezionali. Ad esempio, per indicizzare 10 miliardi di pagine web (da circa 20 KB l'una), un *full scan* sequenziale su un disco rigido tradizionale o l'estrazione a blocchi nella memoria RAM (come nel classico *Data Mining*) richiederebbe un tempo inaccettabile.

Occorreva passare da un'architettura a nodo singolo (*single-node architecture*) a un approccio fortemente distribuito.

### 1.4.2 Il principio della Data Locality

Il concetto base sviluppato da Google per i framework Big Data è la **Data Locality**.

**Definizione 1.1** (Data Locality)**.** Invece di inviare enormi quantità di dati attraverso la rete verso i server che contengono il codice da eseguire, si invia il codice (che è di dimensioni molto ridotte) direttamente ai server in cui i dati sono fisicamente memorizzati.

Inviare troppi dati sulla rete crea dei colli di bottiglia (*network bottleneck*) che rendono l'applicazione lentissima.

### 1.4.3 Architettura dei Data Center e Scalabilità

I sistemi Big Data utilizzano **commodity hardware**, ovvero server standard, economici e facilmente sostituibili in caso di guasto. Questi server sono raggruppati in rack e interconnessi tramite switch di rete.

*Osservazione* 1.3. **High-Performance Computing (HPC) vs Big Data Framework** L'HPC è indicato per compiti intensivi dal punto di vista computazionale (es. calcolare miliardi di operazioni in virgola mobile al secondo su un set di dati relativamente piccolo) e si basa sulla *scalabilità verticale*. I framework Big Data (come Hadoop o Spark) sono invece adatti a carichi di lavoro **data-intensive** e si basano sulla **scalabilità orizzontale**, dove la metrica principale è quanti dati il sistema riesce ad analizzare in un certo lasso di tempo.

### 1.4.4 Le sfide tecniche

Secondo le slide, le principali sfide affrontate dai framework Big Data sono:

- Calcolo distribuito e parallelizzazione;
- Storage distribuito di dataset di dimensioni enormi (Terabyte, Petabyte);
- Gestione dei guasti dei nodi (*Node failure management*);
- Colli di bottiglia della rete (*Network bottleneck*);
- Formati di input eterogenei (*Data diversity & heterogeneity*).

Il vantaggio dell'utilizzo di framework come Spark è che lo sviluppatore non deve occuparsi direttamente dello scheduling dell'applicazione, della gestione della rete o dei fallimenti hardware, poiché vengono gestiti automaticamente dal framework.

### 1.4.5 Il problema tipico nei Big Data

Non tutti i problemi possono essere risolti efficientemente con un approccio Big Data (ad esempio, quando è necessaria una visione "globale" immediata dei dati per prendere una decisione). Inoltre, le funzioni applicate ai dati distribuiti devono soddisfare specifici vincoli matematici: devono essere **commutative** e **associative**.

Un problema tipico che ben si adatta a questi framework segue questo schema operativo:

1. Iterare su una quantità enorme di record/oggetti;
2. Estrarre l'informazione di interesse da ogni record (*Map*);
3. Aggregare i risultati intermedi (*Reduce*);
4. Generare l'output finale / risultato globale.

# Capitolo 2

# Big Data Architectures

## 2.1 Introduzione alle Architetture Big Data

Un'architettura Big Data è progettata per gestire l'ingestione, l'elaborazione e l'analisi di dati che risultano troppo grandi o complessi per i sistemi di database tradizionali.

Le soluzioni Big Data tipicamente coinvolgono una o più delle seguenti tipologie di carico di lavoro (workload):

- **Batch processing** di sorgenti Big Data *at rest* (dati a riposo/archiviati).
- **Real-time processing** di Big Data *in motion* (dati in movimento/streaming).
- **Interactive exploration** (esplorazione interattiva) dei Big Data.
- **Predictive analytics e machine learning**.

### 2.1.1 Quando utilizzare un'architettura Big Data

È opportuno considerare l'adozione di un'architettura Big Data quando si ha la necessità di:

- Memorizzare ed elaborare dati in **volumi troppo ampi** per un database tradizionale.
- Trasformare **dati non strutturati** per finalità di analisi e reportistica.
- Catturare, elaborare e analizzare **flussi illimitati di dati (unbounded streams)** in tempo reale o con bassa latenza (low latency).

*Osservazione* 2.1 (Nota del professore: Scegliere lo strumento giusto). Il professore sottolinea un aspetto fondamentale per un ingegnere: è cruciale utilizzare la soluzione corretta per il problema specifico che si deve risolvere. Non bisogna adottare un framework Big Data solo perché "lo fanno tutti" o perché è una tecnologia di tendenza, specialmente se i dati da gestire sono di piccole dimensioni. Utilizzare un'architettura complessa quando non è necessaria porta unicamente a uno spreco di tempo e di risorse computazionali.

## 2.2 La Lambda Architecture

L'architettura Big Data più frequentemente utilizzata (secondo il professore, presente in circa il 99% dei sistemi basati su framework Big Data) è la **Lambda Architecture**. È stata proposta per la prima volta da Nathan Marz nel 2011 e, seppur basata su idee molto semplici, risulta estremamente efficace ed efficiente.

### 2.2.1 Definizioni a confronto

Per comprendere appieno questa architettura, è utile analizzare tre diverse definizioni fornite in letteratura e nel settore industriale:

**Definizione 2.1** (Definizione #1 - Nathan Marz)**.** I database NoSQL (come Cassandra) ottengono la loro scalabilità offrendo un **modello di dati molto più limitato** rispetto al classico SQL, rendendo complessa l'adattabilità delle applicazioni. Inoltre, essendo database mutabili, **non sono tolleranti agli errori umani (human-fault tolerant)**. La Lambda Architecture nasce dall'uso intelligente e congiunto di diversi strumenti (sistemi di calcolo su larga scala e database) per produrre sistemi scalabili per problemi di dati arbitrari, garantendo la tolleranza agli errori umani con il minimo della complessità.

**Definizione 2.2** (Definizione #2 - Databricks)**.** La Lambda Architecture è un modo per elaborare quantità massive di dati fornendo accesso sia a metodi di **batch-processing** che di **stream-processing** attraverso un **approccio ibrido**. È utilizzata per risolvere il problema del calcolo di funzioni arbitrarie.

**Definizione 2.3** (Definizione #3 - Wikipedia)**.** È un'architettura di elaborazione dati progettata per gestire quantità massive di dati sfruttando i vantaggi sia dei metodi batch che stream. Cerca di **bilanciare latenza, throughput e tolleranza ai guasti (fault-tolerance)** in questo modo:

- Il *batch processing* fornisce viste accurate e complete sui dati archiviati (batch data).
- Lo *stream processing* in tempo reale fornisce viste sui dati online.

Gli output di queste due viste vengono poi uniti (joined) prima della presentazione finale all'utente.

### 2.2.2 Il Modello dei Dati nella Lambda Architecture

La Lambda Architecture si basa fortemente su un modello di dati specifico: utilizza una **sorgente dati immutabile e append-only** che funge da sistema di registrazione (system of record).

È pensata per l'ingestione e l'elaborazione di **eventi dotati di timestamp**, i quali vengono accodati (appended) agli eventi esistenti piuttosto che sovrascriverli. Lo stato del sistema viene determinato dal naturale ordinamento temporale dei dati.

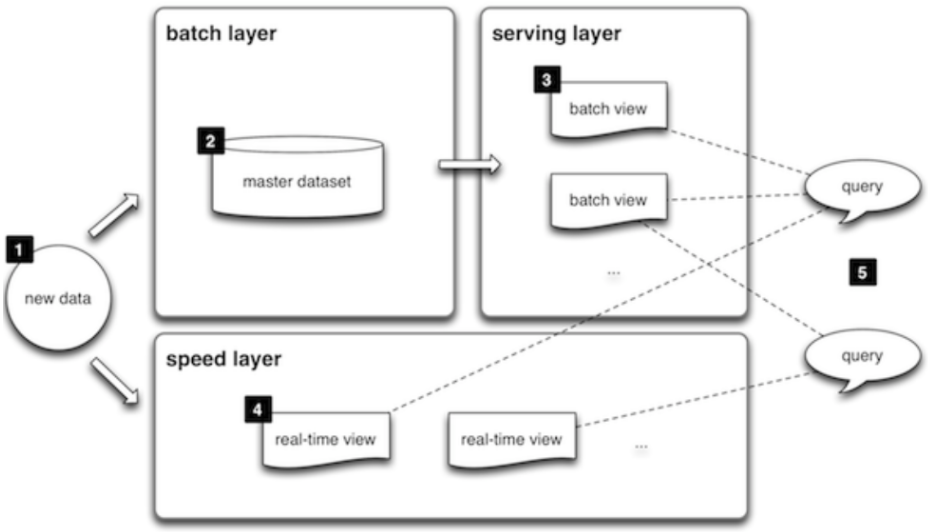



Figura 2.1: Struttura e livelli della Lambda Architecture.

### 2.2.3 Requisiti dell'Architettura

La Lambda Architecture è progettata per soddisfare i seguenti requisiti fondamentali:

- **Tolleranza ai guasti (Fault-tolerance):** Deve resistere sia ai guasti hardware (hardware failures) sia agli errori umani (human errors).
- **Supporto per molteplici casi d'uso:** Deve gestire sia interrogazioni a bassa latenza sia aggiornamenti complessi sui dati storici.
- **Capacità di Linear Scale-out:** Deve poter scalare orizzontalmente in modo lineare.

*Osservazione* 2.2 (Implementazione dello Scale-out). Come specificato dal professore, la scalabilità "scale-out" (ovvero l'aggiunta di nuovi nodi al cluster anziché il potenziamento del singolo nodo) viene implementata utilizzando framework e tecnologie progettate nativamente per questo scopo, come i database NoSQL e i framework Hadoop o Spark. Quando si processano dati nel livello batch, si effettua tipicamente un *full scan* (scansione completa) dell'intera collezione di dati in input, analizzando un record alla volta per ottenere il risultato finale.

# Capitolo 3

# Hadoop e MapReduce

## 3.1 Introduzione al Framework Hadoop

Il framework Hadoop, pur essendo stato proposto molti anni fa, introduce concetti di base che rimangono fondamentali nel mondo dell'elaborazione dei Big Data. I componenti principali di Hadoop includono un'infrastruttura di elaborazione distribuita, interamente basata sul paradigma di programmazione MapReduce, e il file system distribuito HDFS (Hadoop Distributed File System). Inoltre, all'ecosistema Hadoop sono associati molti altri progetti paralleli, come Spark e Hive (un database relazionale distribuito che sfrutta applicazioni MapReduce).

L'infrastruttura Hadoop è caratterizzata da alcune proprietà fondamentali:

- **Astrazione ad alto livello (Separazione del *What* dal *How*):** Hadoop permette ai programmatori di concentrarsi esclusivamente su *cosa* calcolare (implementando il paradigma MapReduce), mentre l'infrastruttura di Hadoop gestisce automaticamente il *come*, occupandosi di tutte le complessità dei sistemi distribuiti, come lo scheduling dei task e la sincronizzazione.
- **Tolleranza ai guasti (Fault-tolerance):** I fallimenti dei singoli server (nodi) o dei task vengono gestiti automaticamente dal sistema Hadoop. Ad esempio, per evitare "single points of failure", vengono mantenute delle copie di backup (shadow copies/repliche) su server diversi che subentrano immediatamente nel caso in cui un nodo principale smetta di funzionare.
- **Alta disponibilità (High availability):** Il file system HDFS garantisce che lo storage distribuito sia sempre altamente disponibile e resiliente agli errori.

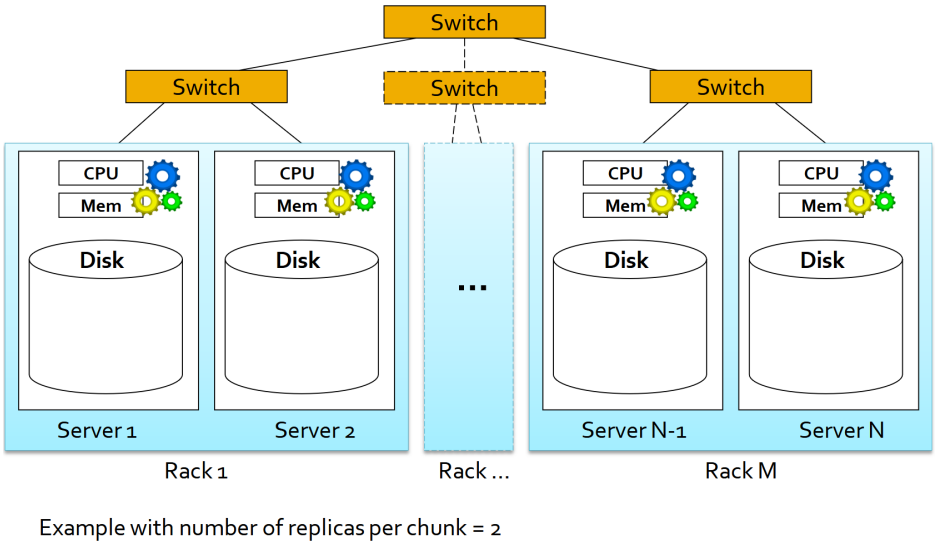



Figura 3.1: Componenti fisici principali e gerarchia di rete in un cluster Hadoop.

## 3.2 Il Principio della Data Locality

Nei sistemi distribuiti tradizionali (come gli ambienti HPC), l'approccio standard consiste nello spostare i dati verso i nodi di calcolo (server). Tuttavia, quando si ha a che fare con i Big Data, questo approccio non è scalabile per processare Terabyte di dati a causa dei severi limiti di larghezza di banda della rete.

Hadoop inverte questo paradigma muovendo il codice verso i dati. Il codice dell'applicazione (che pesa solitamente pochi Kilobyte) viene spedito, copiato ed eseguito direttamente sui server fisici in cui sono materialmente memorizzati i chunk (blocchi) di dati. Questo approccio fondamentale prende il nome di **Data Locality** e minimizza l'uso della rete.

## 3.3 Campi di Applicazione di MapReduce

Hadoop e MapReduce sono stati progettati esplicitamente per applicazioni data-intensive e processi di tipo Batch, ovvero per elaborazioni che richiedono (quasi sempre) una scansione completa di tutti i dati di input.

- **Problemi ideali per MapReduce:** MapReduce è eccellente quando si ha la necessità di iterare e scansionare enormi moli di record per estrarne informazioni. Esempi tipici includono:
  - Lettura ed elaborazione dell'intero Web (es. per il calcolo del PageRank).
  - Lettura ed elaborazione di interi grafi sociali (es. per identificare sottografi connessi e suggerire potenziali amici, noto come Link Prediction).
  - Analisi di log di grandi dimensioni (es. tracce di rete, dati provenienti da smart-meter e sensori di smart-city).
- **Limitazioni:** Hadoop/MapReduce non è la panacea per tutti i problemi legati ai Big Data. Non è adatto per:
  - Analisi di flussi di dati in streaming o elaborazioni in real-time.
  - Problemi di natura strettamente ricorsiva.

– Problemi prettamente iterativi (sebbene sia possibile implementare algoritmi come il PageRank con MapReduce, il framework non è nativamente ottimizzato per cicli di iterazione continui sugli stessi dati).

## 3.4 Il Paradigma di Programmazione MapReduce

Un programma Hadoop è strutturato in una sequenza rigorosa di operazioni basata su due macro-funzioni (building blocks): **Map** e **Reduce**.

*Osservazione* 3.1. Un singolo *job* è formato unicamente da una fase di Map seguita da una fase di Reduce. A differenza di altri sistemi (come il database NoSQL MongoDB), non è possibile concatenare fasi spurie come "Map-Reduce-Reduce" all'interno di uno stesso job. Se un task è molto complesso, deve essere suddiviso in più applicazioni MapReduce sequenziali, dove l'output del primo job diventa l'input del job successivo.

### 3.4.1 Struttura Dati: Coppie Chiave-Valore

La struttura dati di base in MapReduce è la coppia chiave-valore (*key-value pair*). Sia l'input che l'output di un programma MapReduce sono sempre rappresentati da liste di coppie chiave-valore. Le chiavi e i valori possono essere tipi di dato primitivi (come interi, float, stringhe) o anche strutture dati quasi arbitrarie definite appositamente dallo sviluppatore. Per poter elaborare file testuali o altre tipologie di file, esistono classi specifiche in Hadoop incaricate di leggere i dati grezzi e trasformarli in un insieme di coppie chiave-valore di input.

### 3.4.2 Le Fasi di Elaborazione

Ogni applicazione MapReduce è divisa internamente in tre fasi logiche:

1. **Fase di Map:** La funzione di map viene applicata a ciascun elemento del dataset di input ed emette un insieme di nuove coppie $(key, value)$ intermedie. Lo sviluppatore scrive questa funzione per trasformare logicamente il dato di input in base al task da risolvere.

2. **Fase di Shuffle and Sort:** È la fase intermedia che si occupa di raggruppare e ordinare tutto l'output della fase di map raggruppandolo per chiave.

   *Osservazione* 3.2. Questa fase compie sempre la stessa operazione (mettere insieme tutti i valori associati alla stessa chiave) e non deve essere implementata dallo sviluppatore, in quanto è fornita in modo automatico e altamente ottimizzato dal framework Hadoop.

3. **Fase di Reduce:** La funzione di reduce ($r$) può essere vista come un'operazione di aggregazione definita dal programmatore. Viene invocata esattamente una volta per ogni *chiave distinta* e si occupa di aggregare l'intera lista di valori associati a quella specifica chiave. Poiché ogni gruppo di coppie aventi la stessa chiave è totalmente indipendente dagli altri, la fase di reduce viene eseguita in isolamento ed è intrinsecamente parallelizzabile, emettendo le coppie chiave-valore che costituiranno il risultato finale.

## 3.5 Esempio Pratico: Il Problema del Word Count

**Esempio 3.1** (Word Count)**.** Il problema del conteggio delle parole ("Word Count") rappresenta il classico esempio "Hello World" per spiegare il funzionamento delle applicazioni MapReduce. L'obiettivo è analizzare un grande file testuale in input (visto come una lista di parole) per contare il numero totale di occorrenze di ciascuna parola presente nel documento.

1. **Funzione di Map:**

La funzione di map analizza il testo di input. Per ogni parola incontrata, emette semplicemente una coppia chiave-valore dove la chiave è la parola stessa e il valore associato è 1.

$$\text{Map}(\text{input}) \rightarrow (\text{word}, 1)$$

2. **Shuffle and Sort:**

Il sistema riceve tutti i risultati della fase di map e raduna automaticamente tutti gli 1 generati per ogni singola parola.

3. **Funzione di Reduce:**

La funzione di reduce viene chiamata su una singola parola accompagnata dalla lista di tutti i valori a essa associati (che in questo caso sono tutti 1). La funzione somma semplicemente tutti i valori dell'array e restituisce una singola coppia chiave-valore contenente la parola e il suo numero globale di occorrenze.

$$\text{Reduce}(\text{word}, [1, 1, \ldots, 1]) \rightarrow (\text{word}, \text{numero totale di occorrenze})$$

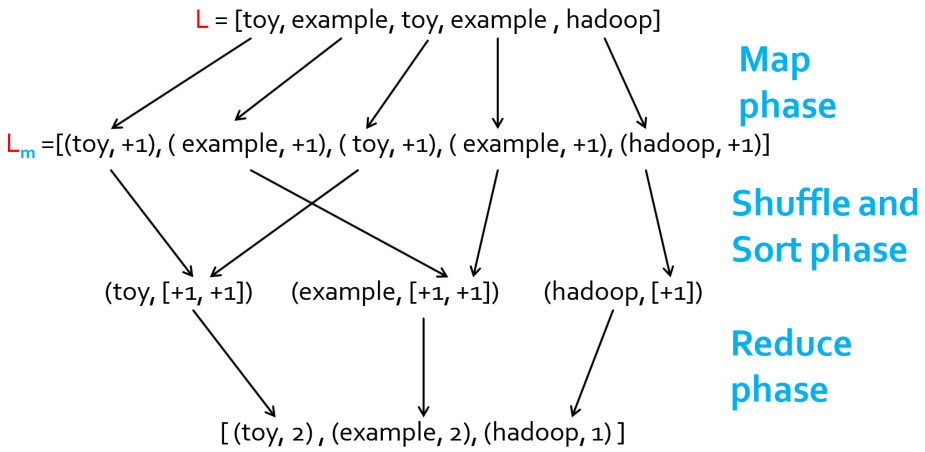



Figura 3.2: Esempio di applicazione MapReduce per il problema del Word Count.

# Capitolo 4

# Sviluppo di applicazioni MapReduce in Hadoop

Questo capitolo illustra come implementare programmi MapReduce utilizzando il framework Hadoop. L'obiettivo principale di Hadoop è astrarre la complessità dei sistemi distribuiti: il progettista o sviluppatore deve concentrarsi unicamente sulla definizione logica delle funzioni di *Map* e *Reduce*, mentre il framework gestisce autonomamente l'esecuzione distribuita, la parallelizzazione, la fase di *shuffle and sort*, lo scheduling dei task e la sincronizzazione.

Le API di MapReduce in Hadoop sono scritte in Java. Sebbene in framework più recenti come Spark sia diffuso l'utilizzo di Python, per Hadoop la scrittura nativa e più performante avviene in Java.

## 4.1 Architettura di un'applicazione MapReduce e Terminologia

Un programma Hadoop MapReduce è costituito da tre componenti logiche fondamentali, ciascuna implementata tramite una specifica classe Java:

- **Driver**: la classe che configura l'applicazione e ne gestisce il flusso di esecuzione.
- **Mapper**: la classe responsabile della fase di *map*.
- **Reducer**: la classe responsabile della fase di *reduce*.

Per comprendere a fondo il funzionamento del framework, è cruciale chiarire la terminologia utilizzata, in particolare la distinzione tra classi e istanze (oggetti):

- **Job**: rappresenta l'esecuzione (o *run*) completa del programma MapReduce su un determinato dataset.
- **Task**: è l'esecuzione di un singolo Mapper (Map task) o di un singolo Reducer (Reduce task) su una porzione specifica di dati. In un Job vi sono solitamente decine o centinaia di task eseguiti in parallelo.
- **Driver, Mapper, Reducer (senza il suffisso "class")**: indicano le singole *istanze* (oggetti) delle rispettive classi create a runtime dal sistema. Ad esempio, il sistema crea numerosi oggetti Mapper, ognuno per analizzare una porzione diversa di dati.
- **Input split**: è una porzione di dimensione fissa dei dati di input. Logicamente, il file di input viene diviso in "split". Generalmente, uno split corrisponde più o meno alla

dimensione di un blocco di HDFS (es. 128 MB). La corrispondenza non è perfetta al 100% perché lo split deve tenere conto dei record (es. righe di testo) che iniziano in un blocco e finiscono nel blocco successivo.

### 4.1.1 Il livello di parallelizzazione

Il numero di istanze della classe Mapper (i *Mapper task*) creati dal sistema è **esattamente uguale al numero di input split**. Se un file è diviso in 10 blocchi su HDFS, Hadoop creerà 10 istanze del Mapper, assegnando ciascuna a un server in cui risiede fisicamente quel blocco di dati per evitare trasferimenti di rete (concetto di *data locality*). Questa rappresenta la massima parallelizzazione ottenibile senza spostare i dati.

## 4.2 Componenti e Flusso dei Dati

### 4.2.1 Il Driver

Il Driver contiene il metodo `main()`, che funge da punto di ingresso (entry point) dell'applicazione. Viene eseguito sulla macchina client (non sui nodi del cluster) e ha il compito di:

- Leggere gli argomenti passati da riga di comando.
- Configurare il Job (definire i percorsi di input/output, i formati, e i tipi di dato previsti).
- Dichiarare le classi utilizzate per il Mapper e il Reducer.
- Sottomettere il Job al cluster Hadoop e attenderne il completamento.

### 4.2.2 Il Mapper

Il Mapper "implementa" la fase di map e viene eseguito direttamente sui nodi del cluster. All'interno della classe, il metodo fondamentale è `map(...)`. Esso riceve in input una coppia $(chiave, valore)$ letta dallo split e viene **invocato una volta per ogni singola coppia**. Analizza i dati e produce in output un insieme (da 0 a N) di coppie intermedie $(chiave, valore)$. I risultati intermedi prodotti dal Mapper **non vengono salvati su HDFS**, ma nel file system locale (disco fisso) del server in cui il Mapper è in esecuzione, al fine di velocizzare le operazioni ed evitare la latenza di replicazione di HDFS.

### 4.2.3 Il Reducer e il partizionamento dei dati (Shuffle and Sort)

Tra la fase di map e la fase di reduce, il sistema raggruppa tutti i risultati intermedi. I dati vengono ordinati in base alla chiave e inviati attraverso la rete ai nodi che eseguono le istanze del Reducer. I dati vengono distribuiti applicando una funzione di *hash* sulla chiave: tutte le coppie che condividono la stessa chiave finiranno inevitabilmente alla stessa istanza del Reducer.

*Osservazione* 4.1 (La scelta del numero di Reducer). Mentre il numero di Mapper è scelto automaticamente dal sistema in base al numero di split, **il numero di Reducer deve essere esplicitamente specificato nel Driver dallo sviluppatore**. Il sistema non può sapere a priori quante chiavi distinte produrrà la fase di map. Lo sviluppatore deve impostare un numero ragionevole per parallelizzare la fase di reduce.

- *Assegnare troppi Reducer*: Se si impostano 200 Reducer per un problema che genera solo 100 chiavi distinte (es. 100 parole uniche), il sistema creerà 200 istanze. Di queste, 100 rimarranno vuote e verranno distrutte subito, sprecando risorse e causando un sovraccarico (overhead) inutile per lo scheduling.
- *La funzione di Hash e lo sbilanciamento (Data Skew)*: La funzione di hash standard cerca di distribuire le chiavi uniformemente tra i Reducer. Tuttavia, se una chiave ha una frequenza anomala (es. la parola "Paolo" compare milioni di volte, mentre "Luca" solo 4 volte), il Reducer a cui viene assegnato "Paolo" diventerà un collo di bottiglia e impiegherà molto più tempo degli altri. In questi rari casi, lo sviluppatore può definire una funzione di partizionamento personalizzata.

Il metodo `reduce(...)` del Reducer viene **invocato una sola volta per ogni chiave distinta**, ricevendo come input la chiave e la lista iterabile di tutti i valori a essa associati. L'output finale prodotto dal Reducer viene, questa volta, scritto nel distributed file system (es. HDFS). Poiché HDFS replica ogni blocco (di default 3 copie) per tolleranza ai guasti, questa fase di scrittura finale genererà traffico di rete.

Figura 4.1: Flusso dei dati in MapReduce: generazione degli input split, elaborazione nei Mapper sui nodi locali, fase di shuffle and sort attraverso la rete, ed esecuzione dei Reducer.

## 4.3 Tipi di Dato (Data Types) in Hadoop

I programmi Hadoop non utilizzano i tipi base nativi di Java (come `String`, `int`, `long`, `float`) come chiavi o valori direttamente nelle firme dei metodi `map` e `reduce`. Per questioni di efficienza nel trasferimento dei dati sulla rete (serializzazione/deserializzazione), Hadoop definisce delle classi *wrapper* specifiche, dette **Writable**:

- `org.apache.hadoop.io.Text`: corrisponde a `String`.
- `org.apache.hadoop.io.IntWritable`: corrisponde a `Integer`.
- `org.apache.hadoop.io.LongWritable`: corrisponde a `Long`.
- `org.apache.hadoop.io.FloatWritable`: corrisponde a `Float`.

Ogni classe utilizzata come **valore** deve implementare l'interfaccia `Writable`. Ogni classe utilizzata come **chiave** deve implementare l'interfaccia `WritableComparable`. La chiave non solo deve essere serializzata, ma deve anche poter essere *comparata* e *ordinata* durante la fase cruciale di shuffle and sort, affinché le chiavi identiche vengano raggruppate.

## 4.4 Formati di Input e Output

Prima che i dati arrivino al Mapper, il file su HDFS deve essere diviso e tradotto in coppie chiave-valore. Questo è gestito dalla classe astratta `InputFormat`, che valida il formato del file, divide i dati in Input Split e fornisce un `RecordReader` per l'estrazione delle coppie.

Esistono implementazioni predefinite di `InputFormat` molto utilizzate, specialmente per i file testuali:

- `TextInputFormat`: È il formato standard per i file di testo. Il file viene diviso per righe (usando il ritorno a capo).

  - **Chiave**: l'offset (posizione in byte) della riga all'interno del file. Il tipo di dato è `LongWritable`.
  - **Valore**: il contenuto dell'intera riga. Il tipo di dato è `Text`.

  Spesso, all'interno del metodo map, l'offset (la chiave) viene ignorato perché non contiene informazioni semanticamente utili per l'analisi.

- `KeyValueTextInputFormat`: Utilizzato per file in cui ogni riga contiene già implicitamente una coppia chiave-valore, divisa da un separatore (di default una tabulazione `\t`).
  - **Chiave**: il testo precedente al separatore. Il tipo di dato è `Text`.
  - **Valore**: il testo successivo al separatore. Il tipo di dato è `Text`.

*Osservazione* 4.2 (Attenzione ai tipi di dato con KeyValueTextInputFormat). Durante la lezione il professore ha sottolineato un errore comune: con `KeyValueTextInputFormat`, **sia la chiave che il valore sono di tipo `Text`**. Se il testo precedente al separatore è un numero (es. `10125\tMister John`), la chiave generata sarà l'oggetto `Text` contenente la stringa "10125", non un intero. Bisogna prestare attenzione quando si scrive il codice del Mapper, per evitare errori di *type mismatch*.

- `SequenceFileInputFormat`: Un formato per file binari sequenziali, ottimizzato per framework Big Data.

Figura 4.2: Processo di acquisizione dei dati: l'InputFormat definisce gli split e il RecordReader genera le coppie chiave-valore passate al Mapper.

Parallelamente, la classe `OutputFormat` (es. `TextOutputFormat`) definisce come salvare i risultati su HDFS. Il formato di default crea file di testo in cui la chiave e il valore sono separati da un tab, rendendoli perfettamente leggibili da un successivo job configurato con `KeyValueTextInputFormat`.

## 4.5 Struttura base del codice Hadoop: Word Count Example

Analizziamo ora la struttura del codice Java attraverso il classico problema del conteggio delle parole (Word Count). L'obiettivo è calcolare le occorrenze di ogni parola in un file testuale.

### 4.5.1 Implementazione del Driver

Il Driver eredita da `Configured` e implementa `Tool`. Lo sviluppatore si concentra sul metodo `run()`, dove imposta il lavoro.

Listing 4.1: Struttura del Driver per il Word Count

```
1 public class WordCount extends Configured implements Tool {
2     @Override
3     public int run(String[] args) throws Exception {
4         int numberOfReducers = Integer.parseInt(args[0]);
5         Path inputPath = new Path(args[1]);
6         Path outputDir = new Path(args[2]);
7 
8         Configuration conf = this.getConf();
```

```
        Job job = Job.getInstance(conf);
        job.setJobName("WordCounter");

        // Impostazione dei path
        FileInputFormat.addInputPath(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputDir);

        // Classi per i formati
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);

        // Collegamento delle classi e del Driver
        job.setJarByClass(WordCount.class);
        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);

        // Tipi di output del MAPPER (fondamentale specificarli)
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);

        // Tipi di output del REDUCER
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        // Numero di reducer desiderati
        job.setNumReduceTasks(numberOfReducers);

        return job.waitForCompletion(true) ? 0 : 1;
    }

    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new Configuration(), new WordCount(), args);
        System.exit(res);
    }
}
```

Il metodo `setMapOutputKeyClass` e il corrispettivo per il value sono essenziali: il sistema non può ricavare automaticamente questi tipi generici a runtime (per via della *type erasure* in Java). Non è invece necessario specificare esplicitamente i tipi in input al Reducer, poiché corrispondono esattamente all'output del Mapper.

### 4.5.2 Implementazione del Mapper

La classe estende `Mapper<ChiaveIn, ValoreIn, ChiaveOut, ValoreOut>`. Per l'input file testuale di default, le chiavi in ingresso sono `LongWritable` (offset) e i valori `Text` (riga di testo).

Listing 4.2: Struttura del Mapper per il Word Count

```
class WordCountMapper extends Mapper<
    LongWritable, Text, // Input: Offset riga, Contenuto riga
    Text, IntWritable> { // Output: Parola, Contatore (1)

    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        // Spezza la riga (value) in parole usando gli spazi come delimitatore
```

```
        String[] words = value.toString().split("\\s+");

        for(String word : words) {
            String cleanedWord = word.toLowerCase();
            // Emette (parola, 1) tramite l'oggetto context
            context.write(new Text(cleanedWord), new IntWritable(1));
        }
    }
}
```

L'oggetto `Context` rappresenta la connessione con l'esecuzione nel cluster e mette a disposizione il metodo `write` per emettere le coppie prodotte.

### 4.5.3 Implementazione del Reducer

La classe estende `Reducer<ChiaveIn, ValoreIn, ChiaveOut, ValoreOut>`. Nel nostro esempio, l'input riceve l'output del Mapper (`Text`, `IntWritable`).

Listing 4.3: Struttura del Reducer per il Word Count

```
class WordCountReducer extends Reducer<
    Text, IntWritable, // Input: Parola, lista di conteggi (1)
    Text, IntWritable> { // Output: Parola, Somma totale

    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int occurrences = 0;

        // Scorre la lista di valori per quella specifica chiave e li somma
        for (IntWritable value : values) {
            occurrences = occurrences + value.get();
        }

        // Emette la parola e la sua frequenza totale
        context.write(key, new IntWritable(occurrences));
    }
}
```

## 4.6 Ottimizzazione delle Prestazioni: Il Combiner

Nel problema Word Count appena visto, se un Mapper trova la parola "Hadoop" mille volte nel suo blocco, produrrà mille coppie `(Hadoop, 1)` che verranno interamente inviate sulla rete verso il Reducer. Per limitare questa congestione, Hadoop permette di utilizzare un **Combiner** (o mini-reducer locale).

Il Combiner esegue una pre-aggregazione dei risultati intermedi all'interno dello stesso server del Mapper, prima della fase di *shuffle*. Così, le mille coppie verranno aggregate localmente nella singola coppia `(Hadoop, 1000)` prima di viaggiare sulla rete.

*Osservazione* 4.3 (Condizioni per l'uso del Combiner). **La funzione applicata dal Combiner deve essere associativa e commutativa** (come la somma o la ricerca del massimo). La media matematica, ad esempio, non lo è. Se si applica una funzione non adeguata, il sistema non restituirà alcun errore, ma il risultato finale sarà semplicemente sbagliato. Inoltre, **l'esecuzione del Combiner non è garantita**. Il framework decide a runtime se eseguirlo o meno (a seconda delle statistiche interne su disco/RAM). Pertanto, la logica dell'applicazione

(e soprattutto del Reducer) deve essere costruita in modo che il risultato finale sia corretto a prescindere dall'esecuzione del Combiner. Questo implica anche che i tipi di dato in output dal Combiner debbano coincidere strettamente con i tipi di dato in output dal Mapper.

### 4.6.1 Perché nel Word Count si sommano i valori anziché contarli?

Durante la lezione è stata posta l'attenzione sul perché nel Reducer (e nel Combiner) del Word Count si effettui la somma dei valori (`occurrences += value.get()`) e non si incrementi semplicemente un contatore (`occurrences++`) per ogni elemento dell'iteratore. Senza il Combiner, ricevendo `(Hadoop, [1, 1, 1])`, iterare e contare con `+1` produrrebbe correttamente il risultato 3. Ma se il Combiner entra in azione, il Reducer potrebbe ricevere `(Hadoop, [2, 1])`. Se si contassero solo gli elementi nell'iterabile, il Reducer restituirebbe erroneamente `(Hadoop, 2)`. Sommandone i valori, invece, si ottiene correttamente 3, garantendo la coerenza del programma indipendentemente dal passaggio per il Combiner.

### 4.6.2 Implementazione e Dichiarazione

Per abilitare il Combiner, bisogna istruire il Driver aggiungendo:

```
job.setCombinerClass(WordCountCombiner.class);
```

Il Combiner estende la classe base astratta `Reducer`. Poiché nella quasi totalità dei casi (come nel Word Count) l'operazione da compiere localmente è identica a quella globale (una somma parziale vs una somma totale), si usa comunemente **la stessa classe del Reducer per implementare il Combiner**. Si può quindi omettere la creazione della classe `WordCountCombiner` e indicare:

```
job.setCombinerClass(WordCountReducer.class);
```

Questo riduce il codice da mantenere ed è la pratica del 99% delle applicazioni Hadoop.

Figura 4.3: Confronto del flusso dati: a sinistra l'invio diretto delle coppie, a destra l'effetto di pre-aggregazione introdotto dall'utilizzo di un Combiner a livello di nodo.

## 4.7 Creazione di Tipi di Dato Personalizzati

Spesso il valore da emettere è un dato complesso (es. due numeri, o un intero e una stringa). È necessario definire tipi di dato (*Data Type*) personalizzati. Se il tipo personalizzato funge da **valore**, deve implementare l'interfaccia `Writable`. Vanno ridefiniti obbligatoriamente i metodi `write(DataOutput out)` (per la serializzazione sulla rete) e `readFields(DataInput in)` (per la deserializzazione nel nodo di destinazione).

Esempio di classe complessa per immagazzinare una somma e un contatore (ad es. per il calcolo della media):

```
public class SumAndCountWritable implements org.apache.hadoop.io.Writable {
    private float sum = 0;
    private int count = 0;

    // Metodi getter e setter omessi per brevita'

    @Override
    public void write(DataOutput out) throws IOException {
        out.writeFloat(sum);
        out.writeInt(count);
```

```
    }

    @Override
    public void readFields(DataInput in) throws IOException {
        sum = in.readFloat();
        count = in.readInt();
    }

    @Override
    public String toString() {
        return "sum=" + sum + ",count=" + count;
    }
}
```

*Osservazione* 4.4 (Ordine di Lettura e Scrittura). Un dettaglio fondamentale, evidenziato dal professore, riguarda l'implementazione dei metodi `write` e `readFields`. L'ordine in cui le variabili vengono serializzate in byte nel metodo `write` deve essere **scrupolosamente identico** all'ordine con cui vengono lette nel `readFields`. La rete invia solo uno stream continuo di byte, senza metadati che descrivono i tipi interni. Se si scrive prima il `float` e poi l'`int`, e nel metodo di deserializzazione si invertissero le chiamate leggendo prima un `int` e poi il `float`, **Java non restituirebbe alcun errore o eccezione a runtime**. I byte verrebbero mal interpretati silenziosamente producendo un risultato totalmente incorretto.

Il metodo `toString()` viene ridefinito perché l'`OutputFormat` di Hadoop (ad esempio il `TextOutputFormat`) invoca questo metodo al momento di scrivere il file finale. Senza di esso, per gli oggetti Java non primari, verrebbe stampato l'identificativo esadecimale del puntatore della cella di memoria, che risulta inutile ai fini dell'applicazione.

Se il tipo personalizzato deve essere usato come **chiave**, esso deve implementare l'interfaccia `WritableComparable`. Oltre ai metodi visti, si devono implementare:

- `compareTo()`: essenziale per lo *shuffle and sort*.
- `hashCode()`: fondamentale affinché la funzione di partizionamento assegni gli oggetti uguali allo stesso Reducer.

## 4.8 Condivisione di Parametri e Contatori

### 4.8.1 Passare parametri al Mapper e al Reducer

La firma dei metodi `map` e `reduce` è fissa. Per passare piccoli parametri configurabili (es. una soglia numerica fornita come argomento riga di comando), si sfrutta l'oggetto `Configuration`. Nel **Driver**, il parametro viene memorizzato sotto forma di stringa **prima** di instanziare il Job:

```
Configuration conf = this.getConf();
conf.set("soglia_temperatura", "50");
// ATTENZIONE: Questo va fatto prima di Job.getInstance(conf)
Job job = Job.getInstance(conf);
```

Nel **Mapper** (o Reducer), il parametro viene recuperato in sola lettura dall'oggetto contesto:

```
String sogliaStr = context.getConfiguration().get("soglia_temperatura");
int soglia = Integer.parseInt(sogliaStr);
```

Tale approccio permette solo la condivisione di poche e piccole stringhe, non per scambiare grandi moli di dati.

### 4.8.2 I Contatori (Counters)

I **Counters** sono interi usati per calcolare statistiche globali (ad es. contare il numero di record malformati in cui manca un campo) senza dover scrivere ulteriori job MapReduce separati. Se il programma incontrasse una riga errata e lanciasse un'eccezione, l'intero Job fallirebbe sprecando tutto il tempo impiegato in precedenza. Identificare, isolare la riga ed incrementare un contatore di errore è un design pattern molto comune.

Si definiscono nel Driver tramite `enum` di Java (es. `COUNTERS.ERROR_COUNT`), inizializzandosi implicitamente a zero. Nei metodi `map` o `reduce`, si richiamano in questo modo:

```
context.getCounter(COUNTERS.ERROR_COUNT).increment(1);
```

*Osservazione* 4.5 (Natura incrementale dei Counter). I metodi `map` e `reduce` in esecuzione sui nodi **possono solo invocare l'incremento** del Counter (`increment(N)`), non possono mai "impostarlo" (`set(10)`). Avendo istanze indipendenti del Mapper in esecuzione concorrente, non si può prevedere uno stato sincronizzato in tempo reale del valore del contatore. Hadoop raccoglierà gli incrementi da tutti i nodi asincronamente e calcolerà il valore globale. Il valore finale definitivo del Counter si potrà quindi estrarre e leggere in sicurezza **solo nel Driver**, al termine dell'esecuzione del Job, sfruttando `job.getCounters().findCounter(...).getValue()`.

## 4.9 Job "Map-Only" e ottimizzazioni avanzate

### 4.9.1 Map-Only Job

Alcuni algoritmi richiedono semplicemente il filtraggio del dataset o una mappatura 1:1, analizzando le righe in totale indipendenza. Un esempio è "restituisci il nome e l'età per gli utenti maggiori di 47 anni". In questi casi, non vi è necessità di aggregare informazioni definendo "gruppi", il che rende superflua la fase di Reduce e l'oneroso trasferimento sulla rete dello *shuffle and sort*. Per implementare un **Map-Only job**, occorre configurare esplicitamente il numero di reducer a zero all'interno del Driver:

```
job.setNumReduceTasks(0);
```

L'output della fase di Map verrà direttamente memorizzato nel distributed file system (con file tipicamente nominati `part-m-00000` anziché `part-r-00000`). Il professore ha inoltre fatto notare che azzerare il numero di reducer nel Driver è **un'eccellente tecnica di debugging**: permette allo sviluppatore di verificare facilmente l'output esatto emesso dal Mapper e isolare eventuali bug prima che passino alla logica del Reducer.

### 4.9.2 In-Mapper Combiner: un passo oltre il Combiner

Mentre il Combiner classico prevede l'istanziamento di un'ulteriore classe dedicata che comunica col Mapper, un'ottimizzazione più avanzata è l'**In-Mapper Combiner**. L'aggregazione viene svolta nativamente *all'interno* del Mapper sfruttando variabili di istanza in memoria RAM e l'intero ciclo di vita della classe, attraverso l'overriding di due metodi aggiuntivi:

- `setup(Context context)`: Viene invocato **una sola volta per ciascuna istanza del Mapper task** (all'inizio, analogo a un costruttore). Viene usato per inizializzare un dizionario associativo/array (*hash map*) in memoria per mantenere i contatori locali di ciascuna chiave.

- `cleanup(Context context)`: Viene invocato **una sola volta al termine** dell'esecuzione del Mapper, per inviare (tramite `context.write()`) i risultati consolidati in rete.

Nel metodo `map()`, lo sviluppatore non emette più le coppie, bensì si limita ad aggiornare le strutture dati pre-inizializzate nel metodo `setup`.

*Osservazione* 4.6 (Rischio dell'In-Mapper Combiner: Memoria limitata). L'uso dell'In-Mapper Combiner annulla il costo di serializzazione e instanziamento logico tra Mapper e Combiner, risultando incredibilmente rapido. Tuttavia, tutto il raggruppamento avviene nella memoria Heap di Java. La memoria RAM standard di un'istanza Mapper ammonta spesso a 1GB. Se il numero di "chiavi distinte" del blocco input elaborato è smisurato (e l'array associativo dovesse assumere dimensioni abnormi), si incorrerà in un inevitabile **Out Of Memory Error**, causando l'uccisione (kill) completa dell'applicazione da parte di Hadoop. L'In-Mapper Combiner andrebbe usato unicamente se c'è garanzia che la struttura statistica da mantenere in memoria occupi spazi gestibili dal framework.

## 4.10 Pattern di Ottimizzazione e Filtraggio in MapReduce

Ora esploreremo alcune tecniche avanzate e design pattern molto comuni per l'ottimizzazione e la gestione dei job MapReduce. In particolare, analizzeremo come implementare un *In-Mapper Combiner* per ottimizzare l'uso della rete e come strutturare un job di puro filtraggio (Map-Only) gestendo il passaggio dinamico di parametri dal Driver ai nodi di calcolo.

## 4.11 Ottimizzazione tramite In-Mapper Combiner (Esercizio 9)

Il problema del Word Count, che abbiamo già affrontato utilizzando una classe Combiner tradizionale, può essere risolto in modo ancora più efficiente utilizzando un approccio chiamato **In-Mapper Combiner**.

L'obiettivo rimane lo stesso: dato in input un file testuale non strutturato, vogliamo calcolare il numero di occorrenze di ciascuna parola. Tuttavia, invece di delegare la pre-aggregazione a una classe Combiner separata, eseguiamo questa operazione direttamente all'interno della classe Mapper, risparmiando l'overhead legato all'istanziazione di ulteriori oggetti e riducendo il traffico di rete.

### 4.11.1 Funzionamento dell'In-Mapper Combiner

In un'implementazione MapReduce standard (o con un Combiner tradizionale), il metodo `map` emette una coppia chiave-valore $(word, 1)$ per ogni singola parola letta. Nel pattern In-Mapper Combiner, invece, modifichiamo il ciclo di vita del Mapper sfruttando i metodi di inizializzazione e chiusura:

1. **Metodo `setup`**: Viene invocato una sola volta per ciascuna *istanza* della classe Mapper (all'avvio della gestione di un input split). Qui inizializziamo una variabile locale, tipicamente un array associativo come una `HashMap`, che utilizzeremo per mantenere le statistiche locali (in questo caso, il conteggio parziale delle parole).

2. **Metodo `map`**: Viene invocato per ogni riga di input. Invece di emettere coppie con il metodo `context.write`, estraiamo le parole e aggiorniamo le frequenze direttamente nella `HashMap` locale in memoria.

3. **Metodo `cleanup`**: Viene invocato una sola volta al termine dell'elaborazione di tutte le righe assegnate a quella specifica istanza del Mapper. Qui iteriamo sulle voci della `HashMap` ed emettiamo le coppie $(word, local_count)$.

In questo modo, l'output generato alla fine della fase di Map è equivalente a quello che produrrebbe un Combiner tradizionale, ma ottenuto in modo più diretto. Il Reducer, di conseguenza, non subisce alcuna modifica e continuerà ad aggregare i conteggi parziali ricevuti per calcolare il risultato globale.

### 4.11.2 Limiti e Gestione della Memoria Principale

Come sottolineato dal professore, sebbene questo approccio sia più efficiente, richiede un'attenzione particolare all'uso della **memoria principale (RAM)**.

Le variabili locali utilizzate per memorizzare le statistiche (la `HashMap`) devono essere sufficientemente piccole da poter risiedere nella memoria principale del server in cui è in esecuzione l'istanza del Mapper. Generalmente, a ogni istanza del Mapper è assegnato circa 1 GB di memoria principale.

È fondamentale chiedersi: *qual è la dimensione massima di questa struttura dati nel caso peggiore?* Nel problema del Word Count, la dimensione massima della `HashMap` corrisponde al **numero massimo di parole distinte** presenti nello split di input gestito dal Mapper. Se analizziamo un documento in inglese, il vocabolario tipico è di circa 20.000 - 30.000 parole distinte. Memorizzare 30.000 stringhe associate a un numero intero richiede uno spazio irrisorio, ben al di sotto del gigabyte a disposizione; pertanto, in questo scenario non si verificheranno errori.

Tuttavia, se il dominio del problema prevedesse, ad esempio, 1 miliardo di parole o chiavi distinte all'interno di un singolo split, la `HashMap` saturerebbe la memoria, causando un `OutOfMemoryError` a runtime. È sufficiente che una sola istanza del Mapper vada in errore di memoria per causare il fallimento (kill) dell'intera applicazione MapReduce, costringendo a ricominciare l'elaborazione da zero. Se si prevede che i dati non possano essere contenuti nella memoria principale, è obbligatorio scartare l'In-Mapper Combiner e utilizzare la soluzione standard basata sulla classe Combiner.

### 4.11.3 Esempio di Dati (Toy Example)

Consideriamo il seguente file di input, presentato nelle slide per l'esercizio 9:

```
Toy example
file for Hadoop.
Hadoop running
example.
```

L'output finale, calcolato combinando i conteggi parziali, sarà la lista delle parole con le rispettive frequenze globali:

```
(example, 2)
(file, 1)
(for, 1)
(hadoop, 2)
(running, 1)
(toy, 1)
```

### 4.11.4 Analisi dell'Implementazione Java

Analizziamo ora il codice Java che implementa l'In-Mapper Combiner per il Word Count.

#### La Classe Driver

Nella classe Driver, la configurazione è pressoché identica a quella di un normale Word Count, ma con una differenza fondamentale: **non viene specificata alcuna classe Combiner**.

```
// Configurazione del job MapReduce
Job job = Job.getInstance(conf);
job.setJobName("Exercise #9 - Word Count - in-mapper combiner");

// Set input/output format
job.setInputFormatClass(TextInputFormat.class);
job.setOutputFormatClass(TextOutputFormat.class);

// Set map class e i tipi di output del map
job.setMapperClass(MapperBigData.class);
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(IntWritable.class);

// Set reduce class e i tipi di output del reduce
job.setReducerClass(ReducerBigData.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

job.setNumReduceTasks(numberOfReducers);
```

#### La Classe Mapper

La classe Mapper contiene il cuore logico dell'In-Mapper Combiner.

```
class MapperBigData extends Mapper<LongWritable, Text, Text, IntWritable> {

    // Variabile locale per mantenere le statistiche
    HashMap<String, Integer> wordsCounts;

    @Override
    protected void setup(Context context) {
        // Inizializzazione della variabile all'avvio del Mapper
        wordsCounts = new HashMap<String, Integer>();
    }

    @Override
    protected void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
        Integer currentFreq;

        // Estrazione delle parole ignorando gli spazi multipli
        String[] words = value.toString().split("\\s+");

        for (String word : words) {
            String cleanedWord = word.toLowerCase(); // Normalizzazione
            currentFreq = wordsCounts.get(cleanedWord);

            if (currentFreq == null) {
```

```
                // Prima occorrenza della parola in questo split
                wordsCounts.put(new String(cleanedWord), new Integer(1));
            } else {
                // Incremento delle occorrenze
                currentFreq = currentFreq + 1;
                wordsCounts.put(new String(cleanedWord), new Integer(currentFreq));
            }
        }
        // NOTA: Nessuna invocazione di context.write() all'interno del map!
    }

    @Override
    protected void cleanup(Context context)
            throws IOException, InterruptedException {
        // Al termine dell'elaborazione, emettiamo i dati pre-aggregati
        for (Entry<String, Integer> pair : wordsCounts.entrySet()) {
            context.write(new Text(pair.getKey()), new IntWritable(pair.getValue()))
                ;
        }
    }
}
```

Come si nota, i dati vengono emessi sulla rete verso i Reducer soltanto nel momento in cui il metodo `cleanup` viene chiamato dal framework Hadoop.

### La Classe Reducer

Come anticipato, la classe Reducer non subisce alcuna alterazione rispetto al Word Count classico, poiché riceve in input coppie dello stesso tipo, pre-aggregate.

```
class ReducerBigData extends Reducer<Text, IntWritable, Text, IntWritable> {
    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
        int occurrances = 0;
        for (IntWritable value : values) {
            occurrances = occurrances + value.get();
        }
        context.write(key, new IntWritable(occurrances));
    }
}
```

## 4.12 Filtraggio dei Dati e Passaggio Dinamico di Parametri (Esercizio 12)

Un altro design pattern fondamentale riguarda il puro filtraggio dei dati basato su condizioni fornite dinamicamente dall'utente. Consideriamo un problema in cui riceviamo in input una collezione di file testuali strutturati contenenti i valori giornalieri di PM10 per un insieme di sensori.

Il formato di ogni riga è:

`sensorId,date\tPM10 value (ug/m3)\n`

L'obiettivo è restituire solo i record in cui il valore di PM10 è strettamente inferiore a una specifica **soglia fornita dall'utente come parametro a riga di comando**.

### 4.12.1 Il Pattern di Filtraggio (Map-Only Job)

Analizzando il problema logico, notiamo che:

- Ogni riga può essere valutata indipendentemente dalle altre.
- Non c'è necessità di aggregare o raggruppare record provenienti da sensori o date differenti.

Quando l'elaborazione non richiede raggruppamento (grouping) né aggregazione, i Reducer sono inutili e rappresentano un overhead di rete e di computazione. La soluzione ottimale è utilizzare un **Map-Only Job**, impostando il numero di task reducer a zero (`job.setNumReduceTasks(0)`).

In un Map-Only job, l'output emesso dal metodo `map` non passa attraverso la fase di shuffle and sort, ma viene scritto direttamente nell'HDFS.

Riguardo al formato dell'output, il professore ha illustrato diverse alternative implementative, tutte ugualmente valide ma con differenze formali:

1. **Chiave e Valore estratti dal testo**: Possiamo emettere come chiave `sensorId,date` e come valore il `PM10`. Questa è la soluzione adottata nel codice finale dell'esercizio.
2. **Chiave vuota e Valore linea intera**: Emettere una stringa vuota come chiave e l'intera riga originale come valore.
3. **NullWritable e Valore linea intera**: È possibile utilizzare `NullWritable` come chiave. Si tratta di un tipo di dato predefinito in Hadoop che rappresenta il valore nullo in forma serializzata. È l'oggetto più piccolo definibile nel framework, persino più leggero di una stringa vuota, e risulta particolarmente utile quando la logica non richiede una vera e propria chiave da associare al valore emesso.

### 4.12.2 Passaggio di Parametri tramite l'Oggetto Configuration

La soglia di filtraggio non è nota a priori (hardcoded), ma viene passata all'applicazione tramite gli argomenti (`args`). Questo pone un problema: la classe Driver riceve gli argomenti, ma è la classe Mapper (che viene eseguita su cluster remoti) a dover applicare il filtro.

Per condividere piccole quantità di informazioni dal Driver ai task (Mapper o Reducer), si utilizza l'oggetto `Configuration`. Il meccanismo prevede la definizione di una **proprietà personalizzata** (personalized property) all'interno del Driver, che i task potranno successivamente recuperare.

Le proprietà personalizzate supportano esclusivamente valori di tipo `String`, per cui anche un valore numerico come 21.5 o 21 dovrà essere scritto e letto come testo, e convertito (cast) solo dal Mapper.

#### Attenzione: L'Ordine di Inizializzazione della Configuration

Il professore ha richiamato l'attenzione su un errore logico estremamente comune e insidioso, dimostrato eseguendo l'applicazione passo-passo.

Se si definisce la proprietà personalizzata *dopo* aver creato l'istanza del `Job`, in questo modo:

```
1 Configuration conf = this.getConf();
2 Job job = Job.getInstance(conf); // Creazione del Job
3 conf.set("maxThreshold", args[2]); // ERRORE! Impostazione successiva
```

Il Mapper **non troverà** la proprietà. Questo accade perché l'invocazione di `Job.getInstance(conf)` crea il Job e contestualmente **genera una copia** dell'oggetto `Configuration`. Le modifiche successive fatte all'oggetto `conf` originale nel Driver non si rifletteranno nella copia passata ai Mapper.

In questo scenario di errore:

- Il metodo `context.getConfiguration().get("maxThreshold")` nel Mapper restituirà `null`.
- La conversione `Float.parseFloat(null)` fallirà in modo silenzioso in background sui nodi.
- L'applicazione terminerà creando la cartella di output, ma questa **risulterà completamente vuota**, senza che sulla console vengano stampati errori o eccezioni visibili (se non si ispezionano i log approfonditi dei container).

La regola fondamentale è: **tutte le proprietà personalizzate devono essere definite sull'oggetto Configuration prima di istanziare l'oggetto Job**.

### 4.12.3 Esempio di Dati

Consideriamo l'input dell'esercizio 12 fornito sulle slide:

```
s1,2016-01-01   20.5
s2,2016-01-01   60.2
s1,2016-01-02   30.1
s2,2016-01-02   20.4
s1,2016-01-03   55.5
s2,2016-01-03   52.5
```

Se passiamo al programma una soglia (*threshold*) pari a 21, l'output conterrà esclusivamente le righe relative ai valori 20.5 e 20.4, essendo gli unici strettamente minori di 21:

```
s1,2016-01-01   20.5
s2,2016-01-02   20.4
```

### 4.12.4 Analisi dell'Implementazione Java

Esaminiamo la soluzione implementativa per questo problema di filtraggio parametrico.

#### La Classe Driver

La classe Driver imposta il parametro e definisce il job come Map-Only. Sfrutta inolte il `KeyValueTextInputFormat`, un input format molto utile quando i dati in input sono separati da un tab (`\t`). Questo formato divide in automatico la stringa prima del tab (la pone nella chiave) e la stringa dopo il tab (la pone nel valore).

```
1 public int run(String[] args) throws Exception {
2     Path inputPath = new Path(args[0]);
3     Path outputDir = new Path(args[1]);
4
5     Configuration conf = this.getConf();
6
7     // 1. DEFINIZIONE DELLA PROPRIETA' (PRIMA DEL JOB)
8     // Inviamo la soglia presa da riga di comando ai mapper
```

```
    conf.set("maxThreshold", args[2]);

    // 2. CREAZIONE DEL JOB (Crea una copia della configurazione)
    Job job = Job.getInstance(conf);
    job.setJobName("Exercise 12");

    FileInputFormat.addInputPath(job, inputPath);
    FileOutputFormat.setOutputPath(job, outputDir);

    job.setJarByClass(DriverBigData.class);

    // Utilizzo del KeyValueTextInputFormat
    job.setInputFormatClass(KeyValueTextInputFormat.class);
    job.setOutputFormatClass(TextOutputFormat.class);

    job.setMapperClass(MapperBigData.class);
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(FloatWritable.class);

    // IMPOSTAZIONE MAP-ONLY JOB
    job.setNumReduceTasks(0);

    return job.waitForCompletion(true) ? 0 : 1;
}
```

### La Classe Mapper

Anche in questo caso viene sfruttato il metodo `setup`. Piuttosto che richiedere e parsare l'oggetto `Configuration` per ogni singola riga analizzata dal metodo `map` (che introdurrebbe un'inefficienza notevole visto che la soglia è costante), il valore viene estratto una volta sola all'avvio del task.

```
class MapperBigData extends Mapper<Text, Text, Text, FloatWritable> {

    float threshold;

    @Override
    protected void setup(Context context) {
        // Recupero la soglia una volta sola per l'intero split
        // Viene letto come stringa e convertito in float
        threshold = Float.parseFloat(context.getConfiguration().get("maxThreshold"))
            ;
    }

    @Override
    protected void map(Text key, Text value, Context context)
            throws IOException, InterruptedException {

        float measure;

        // Il value contiene il valore PM10 grazie al KeyValueTextInputFormat
        measure = Float.parseFloat(value.toString());

        // Filtraggio: se rispetta la condizione, emettiamo il record
        if (measure < threshold) {
            context.write(new Text(key), new FloatWritable(measure));
```

```
24          }
25          // Se non rispetta la condizione, il metodo termina senza emettere nulla.
26      }
27  }
```

Poiché i Reducer sono impostati a 0, l'output emesso da `context.write` nel Mapper viene scritto direttamente in formato testuale all'interno dei file di output in HDFS (ad esempio `part-m-00000`), producendo esattamente il risultato atteso.